

本小节内容

课时 16 作业

课时 16 作业

Description

读取 10 个整型数据 12 63 58 95 41 35 65 0 38 44，然后通过冒泡排序，快速排序，插入排序，分别对该组数据进行排序，输出 3 次有序结果，每个数的输出占 3 个空格

Input

12 63 58 95 41 35 65 0 38 44

Output

输出 3 次有序结果

0 12 35 38 41 44 58 63 65 95

0 12 35 38 41 44 58 63 65 95

0 12 35 38 41 44 58 63 65 95

这道题考察的是冒泡排序，快排排序，插入排序，和我们上课写的代码是类似的，只是需要首先通过 for 循环读取 10 个整型元素，下面直接上代码

答案如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
```

```
typedef int ElemType;
typedef struct {
    ElemType* elem;
    int TableLen;
}SSTable;

void ST_init(SSTable& ST, ElemType A[], int len)
{
    ST.TableLen = len;
    ST.elem = (ElemType*)malloc(sizeof(ElemType)*(ST.TableLen));
    int i;
    srand(time(NULL));
    for (i = 0; i < ST.TableLen; i++)
    {
        //ST.elem[i] = rand() % 100;//这里是方便测试
        ST.elem[i] = A[i];//提交到 OJ 时使用这一句
    }
}

void ST_print(SSTable ST)
{
    for (int i = 0; i < ST.TableLen; i++)
    {
        printf("%3d", ST.elem[i]);
    }
    printf("\n");
}

void swap(ElemType& a, ElemType& b)
{
    ElemType temp;
    temp = a;
    a = b;
    b = temp;
}

void BubbleSort(ElemType A[], int n)
{
    int i, j;
    bool flag;
    for (i = 0; i < n - 1; i++) {
        flag = false;
```

```
for (j = n - 1; j > i; j--) {  
    if (A[j] < A[i])  
        swap(A[j], A[i]);  
    flag = true;  
}  
if (false == flag) {  
    break;  
}  
}
```

// 挖坑法，王道书上使用的方法，最左边作为分割值

```
int Partition(ElemType A[], int low, int high)  
{  
    ElemType pivot = A[low]; //把最左边的值暂存起来  
    while (low < high)  
    {  
        while (low < high && A[high] >= pivot) //让 high 从最右边找，找到比分割值小，循环结束  
            --high;  
        A[low] = A[high];  
        while (low < high && A[low] <= pivot) //让 low 从最左边开始找，找到比分割值大，就结束  
            ++low;  
        A[high] = A[low];  
    }  
    A[low] = pivot;  
    return low;  
}
```

```
void QuickSort(ElemType A[], int low, int high)  
{  
    if (low < high)  
    {  
        int pivotpos = Partition(A, low, high);  
        QuickSort(A, low, pivotpos - 1);  
        QuickSort(A, pivotpos + 1, high);  
    }  
}
```

```
void InsertSort(ElemType* arr, int n)  
{  
    int i, j, insertVal;  
    for (i = 1; i < n; i++) //控制要插入的数  
    {
```

```
insertVal = arr[i]; //先保存要插入的值
//内层控制比较, j 要大于等于 0, 同时 arr[j] 大于 insertVal 时, arr[j] 位置元素往后覆盖
for (j = i - 1; j >= 0 && arr[j] > insertVal; j--)
{
    arr[j + 1] = arr[j];
}
arr[j + 1] = insertVal;
}
```

```
int main()
{
    SSTable ST;
    ElemType A[10];
    for (int i = 0; i < 10; i++) //读取输入的 10 个元素
    {
        scanf("%d", &A[i]);
    }
    //冒泡排序
    ST_init(ST, A, 10); //这里是为了方便自己验证做了随机数
    BubbleSort(ST.elem, 10);
    ST_print(ST);
    //快速排序
    ST_init(ST, A, 10); //每次把读取的标准输入重新放入 ST 中
    QuickSort(ST.elem, 0, 9);
    ST_print(ST);
    //插入排序
    ST_init(ST, A, 10); //每次把读取的标准输入重新放入 ST 中
    InsertSort(ST.elem, 10);
    ST_print(ST);
    return 0;
}
```